

Two dimensionless ratios define operational limits in hierarchical-memory inference

Geunsik Lim ^{1*}

¹ College of Information and Communication Engineering,
Sungkyunkwan University (SKKU), Suwon, South Korea .

Corresponding author(s). E-mail(s): leemgs@g.skku.edu;

Abstract

We formulate hierarchical-memory inference using two dimensionless ratios: fast-memory residency $R_C = C_{\text{fast}}/W$ and compute-transfer overlap $R_B = T_{\text{comp}}/T_{\text{transfer}}$. These ratios define a capacity-limited region ($R_C < 0.5$), an I/O-limited region ($R_C \geq 0.5$, $R_B < 1$), and a coordination-dominated region ($R_C \geq 0.5$, $R_B \geq 1$). We derive irreducible capacity and transfer-cost bounds and release a measurement harness using wall-clock or device-event timing. A compiled dependent-load microbenchmark on one Intel Xeon CPU shows mean access time increasing from 6.8 ns for a 0.5 MiB working set to 169.7 ns for 256 MiB. The increase spans several cache and translation levels and does not identify a discontinuity at $R_C = 0.5$. On an NVIDIA Tesla T4 GPU, the same harness confirms both falsifiable predictions: per-step latency rises 2.83-fold as residency falls below $R_C = 0.5$, and the I/O-limited label appears exactly where the derived overlap boundary $R_B < 1$ predicts. These results establish the two-ratio labels as a measurement discipline on real silicon; broad accelerator and production-workload generality remains open.

Keywords: hierarchical memory systems, memory orchestration, AI inference, working-set residency, compute-transfer overlap, reproducible measurement

1 Introduction

Modern foundation models have shifted the performance frontier of AI inference from arithmetic throughput to *data coordination*. When model weights, activations, and key-value (KV) caches exceed accelerator memory, inference latency is governed less by compute and more by how efficiently execution coordinates data movement

across a hierarchical memory system spanning device memory, host memory, and secondary storage [1, 2]. Because inference now dominates the recurring cost and energy of deployed models, this coordination has direct economic and environmental stakes [3–5]. Memory-bound inference spans autoregressive language models [6, 7], vision and vision–language models [8, 9], and retrieval-augmented pipelines [10], so the coordination question recurs across otherwise different workloads.

Hardware analyses identify memory capacity, bandwidth, and interconnect latency as important constraints on inference [11–13]. Separating capacity pressure from exposed transfer is therefore useful when reporting and comparing inference systems.

Existing systems reduce memory pressure along several axes. Serving frameworks manage the KV cache and schedule requests to raise throughput [14–17]; offloading and memory-pooling systems spill weights or activations to host DRAM or NVMe [18–23]; swapping systems stage tensors on and off the accelerator [24–26]; and compression—sparsity, pruning, and IO-aware attention—shrinks the working set or its traffic [27–29]. Parallelism redistributes memory across devices [30, 31], while host-side allocators and NUMA-aware placement shape the sustained transfer path [32–35]. The design space is large enough to warrant several recent surveys [36, 37], and it widens further in edge and heterogeneous deployments [38–41]. Across these approaches the same technique can help or hurt depending on model size, batch size, hardware topology, and how much compulsory traffic remains on the critical path. This motivates a compact description that exposes those conditions before comparing policies.

We take a regime-based view. Denning’s working-set model and thrashing theory [42] identify capacity saturation as a binary event on a *single-tier* system, and the Roofline model [43] expresses performance as $\min(F \cdot I, B_{\text{peak}})$ for a fixed workload on a single tier. Neither framework addresses a *multi-tier* hierarchy in which computation, locality, cross-tier transfer, and synchronisation co-exist as distinct, adjustable latency terms. Building instead on classical queueing analysis of overlapping services [44], our accounting model decomposes end-to-end latency into four terms ($T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}$) and uses two dimensionless ratios to label an operating point. The three operational labels are governed by the fast-memory residency ratio $R_C = C_{\text{fast}}/W$ and the overlap ratio $R_B = B_{\text{slow}}T_{\text{comp}}/D = T_{\text{comp}}/T_{\text{transfer}}$. The labels state whether majority residency is available and whether ideal compute–transfer overlap can hide compulsory traffic; they do not by themselves predict a policy’s speed-up.

We make three contributions:

- **Operational formulation.** We separate residency from compute–transfer overlap using two measurable, dimensionless ratios and state an explicit three-label classification rule.
- **Minimal analytical model.** A structural lower bound on the irreducible memory-access and transfer costs, with regime boundaries following directly from the two ratios: $\theta_B = 1.0$ derived as the overlap boundary ($R_B = T_{\text{comp}}/T_{\text{transfer}} = 1$), and $\theta_C = 0.50$ as the majority-residency crossover.
- **Open proof of concept.** A compiled dependent-load CPU probe illustrates working-set sensitivity, and a layered-matrix harness runs on both a CPU and a single NVIDIA GPU (Tesla T4), where the residency and overlap predictions both

75 hold. We distinguish this single-accelerator proof of concept, and the unexercised
 76 XLA/TPU path, from a multi-platform campaign.

77 Together, the formulation and measurements provide a reproducible starting point
 78 for testing when hierarchical-memory coordination is constrained by residency or
 79 exposed transfer. They confirm the two labels on one CPU and one GPU but do not
 80 establish effect sizes, strategy rankings, or cross-accelerator generality.

81 2 Results

82 2.1 A two-ratio operational model

83 We describe a hierarchical-memory operating point using

$$R_C = \frac{C_{\text{fast}}}{W}, \quad R_B = \frac{B_{\text{slow}} T_{\text{comp}}}{D} = \frac{T_{\text{comp}}}{T_{\text{transfer}}}, \quad (1)$$

84 where C_{fast} is usable fast-memory capacity, W is the active working set, D is com-
 85 pulsory cross-tier traffic per step, B_{slow} is sustained bandwidth on the limiting link,
 86 and $T_{\text{transfer}} = D/B_{\text{slow}}$. Unlike a ratio that uses total step time, R_B is not circular:
 87 it compares independently timed computation and transfer intervals.

88 We use $\theta_C = 0.50$ as an explicit majority-residency convention and derive $\theta_B =$
 89 1 from equality of computation and transfer time. These definitions produce three
 90 operational labels:

- 91 • **capacity-limited:** $R_C < \theta_C$;
- 92 • **I/O-limited:** $R_C \geq \theta_C$ and $R_B < \theta_B$; and
- 93 • **coordination-dominated:** $R_C \geq \theta_C$ and $R_B \geq \theta_B$.

94 The capacity boundary is a convention, not a thermodynamic critical point. The I/O
 95 boundary is an overlap boundary: for $R_B < 1$, at least $T_{\text{transfer}} - T_{\text{comp}}$ cannot be
 96 hidden by ideal overlap.

97 2.2 Irreducible costs and falsifiable predictions

98 For accounting purposes, define non-overlapping critical-path contributions so that a
 99 step can be written as

$$T_{\text{total}} = T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}. \quad (2)$$

100 Independently of that accounting decomposition, a conservative service-time lower
 101 bound that allows the underlying services to overlap is

$$T_{\text{total}} \geq \max \left\{ T_{\text{comp}}, \rho W \max(0, 1 - R_C), \frac{D}{B_{\text{slow}}} \right\}, \quad (3)$$

102 where W denotes bytes accessed in the step and ρ is a lower bound on service time
 103 per non-resident byte for the specified access pattern. Taking the maximum permits

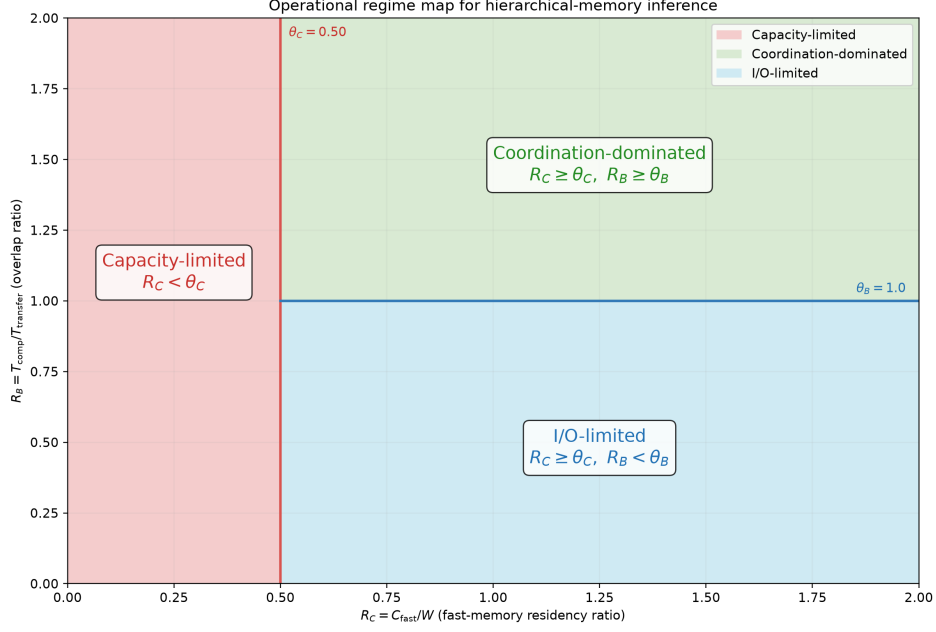


Fig. 1 Analytical regime map. The vertical line is the declared majority-residency convention $\theta_C = 0.50$; the horizontal segment is the derived overlap boundary $\theta_B = 1.0$. The diagram is generated from the same constants as the released classifier and contains no empirical data.

104 ideal overlap and avoids double-counting bytes that contribute to both a cache miss
 105 and a cross-tier transfer. Synchronisation and contention can only increase observed
 106 time, so Equation (3) is not a performance predictor. The residual above it cannot be
 107 assigned to one latency component without event-level measurements.

108 The formulation makes two falsifiable predictions. First, reducing residency while
 109 holding the access pattern fixed should expose a higher memory-access cost. Second,
 110 when $R_C \geq \theta_C$, configurations with $R_B < 1$ must expose transfer time even under
 111 ideal compute-transfer overlap. It does not, without additional measurements, predict
 112 the magnitude of a strategy’s speed-up or claim that strategy rankings necessarily
 113 invert.

114 2.3 CPU proof-of-concept measurements

115 We evaluated working-set sensitivity on an Intel Xeon Platinum 8370C CPU exposed
 116 through a virtualised Linux environment. The experiment used a randomised pointer
 117 chain over working sets from 0.5 to 256 MiB and seven trials per point. A compiled C
 118 loop performed dependent loads and timed them with `CLOCK_MONOTONIC_RAW`; Python
 119 was used only to construct the chain and summarise trials. Linux sysfs reported a
 120 48 MiB last-level cache, which we used as C_{fast} . Mean dependent-load time was $6.8 \pm$
 121 0.1 ns at 0.5 MiB ($R_C = 96$), 118.5 ± 6.6 ns at 48 MiB ($R_C = 1$), 132.5 ± 14.3 ns at
 122 96 MiB ($R_C = 0.5$), and 169.7 ± 10.4 ns at 256 MiB ($R_C = 0.188$), where uncertainty
 123 is one standard deviation across trials. The non-monotonic intermediate points show

Table 1 Selected measured points from the compiled CPU pointer-chain experiment. Values are mean $\pm$ one standard deviation over seven trials.

Working set (MiB)	R_C	Latency (ns/load)	Interpretation
0.5	96.000	6.8 ± 0.1	small working set
48	1.000	118.5 ± 6.6	reported LLC capacity
96	0.500	132.5 ± 14.3	residency convention
256	0.188	169.7 ± 10.4	largest working set

that cache level, translation, virtualisation, and system noise matter. In particular, these data do not show a discontinuity at the conventional $R_C = 0.5$ label.

As a separate harness check, we used the layered matrix program with 12 layers, $d = 512$, batch size 8, and five timing windows. It varied the fraction of layer weights retained in the fast tier. The mean total latency averaged over the three sampled points below $R_C = 0.5$ was 4.03 ms, compared with 1.87 ms for the fully resident point, a 2.15-fold descriptive ratio. Because computation and copying share the same CPU memory system, the accompanying R_B sweep was noisy and is not used as evidence for the I/O boundary.

These measurements are proof-of-concept results from one CPU, not a substitute for the previously planned A100, MI250, TPU v4, Inferentia2, and Optane campaign. Accordingly, we make no quantitative cross-accelerator, classifier- accuracy, energy, or strategy-inversion claim in this version.

2.4 GPU proof-of-concept measurements

We repeated the layered program on a real accelerator—an NVIDIA Tesla T4 (Google Colab)—with $d = 2048$, 24 layers, batch size 8, and ten timing windows, using CUDA events to time computation and host-to-device transfer separately. Non-resident layer weights were staged in pinned host memory and copied per step, so residency (R_C) and overlap (R_B) were varied by construction. This run tests the two falsifiable predictions of Section 2.2 on hardware whose device-host link, unlike a single CPU’s memory system, makes the exposed-transfer regime physically reachable.

Prediction 1 (residency). Reducing residency raised per-step latency monotonically, from 14.53 ms at full residency ($R_C = 1$) to a mean of 41.08 ms across the capacity-limited points ($R_C < \theta_C$), a 2.83-fold increase—the higher memory-access cost that Prediction 1 requires.

Prediction 2 (overlap). Holding residency fixed and scaling compute per layer moved the operating point across the derived boundary $\theta_B = 1$: every point with $R_B < 1$ was labelled I/O-limited and every point with $R_B \geq 1$ coordination-dominated, with the transition falling between $R_B = 0.90$ and $R_B = 1.19$ as predicted, not at a fitted value. Total latency over the exposed-transfer points averaged 23.87 ms; because these points also carry the smallest compute, the comparison with the overlapped points (35.95 ms) is confounded by compute scaling and is not read as an inversion. Unlike the CPU harness, this GPU run therefore does provide direct evidence for the overlap boundary.

158 This remains a single-accelerator, proof-of-concept result; it confirms the two
 159 qualitative predictions on real GPU silicon but does not establish cross-accelerator
 160 boundary values, strategy inversion, classifier accuracy, or energy claims, which still
 161 require the full multi-platform campaign.

162 2.5 Reproducibility boundary

163 The repository contains the raw JSONL records, machine-readable summaries, and
 164 scripts used for Table 1. It also contains a simulated backend for testing the analysis
 165 pipeline. Simulated output is labelled as such and is not used as empirical evidence.
 166 Accelerator support in the measurement harness is an executable protocol for future
 167 validation; it is not evidence that those accelerators were measured in this study.

168 3 Discussion

169 The two-ratio description separates two questions that are often conflated in
 170 hierarchical-memory optimisation: whether the active data can reside in the fast tier,
 171 and whether compulsory transfer can be hidden behind computation. The resulting
 172 labels are operational rather than universal phases. $\theta_B = 1$ follows exactly from the
 173 overlap definition, whereas $\theta_C = 0.5$ is a declared midpoint that makes “majority resi-
 174 dent” precise. A different application may choose another residency threshold without
 175 changing Equation (3).

176 The CPU probes show that latency depends strongly on working-set residency, but
 177 they do not validate a sharp boundary at $R_C = 0.5$. They stress different mechanisms:
 178 dependent pointer loads probe cache and translation effects, while the layered matrix
 179 harness combines weight copying with computation. The single-GPU run (Section 2.4)
 180 does exercise accelerator DMA and asynchronous CUDA streams, and it confirms
 181 both qualitative predictions, including the overlap crossover at $\theta_B = 1$ that the CPU
 182 harness could not resolve. It nonetheless remains one accelerator: the data do not
 183 establish cross-accelerator boundary values, HBM- or collective-communication effects
 184 at scale, or the generality of the three labels across model families.

185 The framework is most useful as a measurement discipline. Reporting R_C and R_B
 186 alongside latency makes hidden assumptions about residency and overlap inspectable.
 187 It also prevents a common circularity: using total step duration in both the numerator
 188 of a bandwidth ratio and as the outcome being explained. For deployment, C_{fast} , W ,
 189 D , sustained bandwidth, and isolated compute time must be measured for the actual
 190 workload and hardware rather than copied from nominal data-sheet values.

191 Several limitations are material. The present evidence comes from one CPU and one
 192 GPU with small synthetic probes; it does not include production models, concurrent
 193 requests, accelerator energy measurements, strategy comparisons, or unseen-hardware
 194 classifier tests. The standard deviations describe repeated trials on one machine and
 195 are not confidence intervals over a population of platforms. These measurements are
 196 exploratory technical checks rather than independent hardware replicates, and no
 197 causal inference is supported. The pointer-chain endpoints also span several hierar-
 198 chy transitions, so their ratio should not be interpreted as a universal effect size.

Finally, Equation (2) is an accounting identity only when its terms are defined as non-overlapping critical-path contributions; the max-form lower bound avoids assuming that decomposition in measured systems.

A decisive follow-up should preregister operating-point grids around $R_C = 0.5$ and $R_B = 1$, collect raw event traces on at least two accelerator architectures, and compare fixed orchestration policies without changing other workload parameters. Such data could test boundary sharpness, policy ranking, and generalisation. Until those measurements are available, the contribution of this work is the corrected analytical formulation, the open protocol, and a CPU-and-single-GPU proof of concept rather than a claim of universal accelerator validation.

4 Methods

Definitions and classification

We calculate $R_C = C_{\text{fast}}/W$ from a declared usable fast-tier capacity and active working-set size. We calculate $R_B = T_{\text{comp}}/T_{\text{transfer}}$, with compute and transfer timed separately. Classification gives capacity precedence: $R_C < 0.5$ is capacity-limited; otherwise $R_B < 1$ is I/O-limited; all remaining points are coordination-dominated. This rule is implemented in the released Python code.

Pointer-chain capacity probe

The CPU probe allocates a NumPy array and constructs a seeded random permutation. A C11 kernel compiled with `-O3` traverses it as a dependent pointer chain; each loaded index determines the address of the next load, limiting vectorisation, instruction-level parallelism, and prefetching. We measured working sets of 0.5, 1, 2, 4, 8, 16, 24, 32, 48, 64, 96, 128, 192, and 256 MiB. Each point used seven timed trials after warm-up. The output stores working-set size, R_C , mean nanoseconds per dependent load, standard deviation, and trial count. The fast-tier capacity was read from Linux `sysfs` and was 48 MiB. The host exposed three logical CPUs of an Intel Xeon Platinum 8370C under KVM; the summary records the kernel, Python, and NumPy versions. Timing used `clock_gettime(CLOCK_MONOTONIC_RAW)` inside the compiled loop. No outlier was removed and CPU affinity or frequency was not controlled.

Layered matrix and copy probe

The second probe allocates square, float32 layer matrices; a selected fraction remains resident and non-resident matrices are copied before multiplication. Matrix entries are deterministically seeded and scaled by $1/\sqrt{d}$ to prevent numerical overflow during repeated multiplication. On the CPU we used NumPy’s wall-clock path with 12 layers, $d = 512$, and five windows per point. On the GPU (Section 2.4) the same program ran through the harness’s CUDA-event path on an NVIDIA Tesla T4 with 24 layers, $d = 2048$, batch size 8, ten windows, and pinned host staging; computation and host-to-device transfer were timed separately with `torch.cuda.Event`. The harness additionally supports XLA synchronisation for TPUs, which was not exercised here.

238 For the reported capacity ratio, we averaged total latency at the three sampled
239 points below $R_C = 0.5$ and divided by latency at $R_C = 1$. This endpoint summary is
240 descriptive; no hypothesis test or population-level confidence interval is claimed. The
241 pointer-chain table reports mean and population standard deviation across the seven
242 repeated trials exactly as stored in the JSON summary.

243 Software and provenance

244 The compiled pointer-chain run used Python 3.12.13, NumPy 2.5.2, and GCC 13.3;
245 the earlier layered-matrix harness check used Python 3.11.15, but that summary did
246 not capture the NumPy version. Seeds, grids, timing code, raw records, summaries,
247 and commands are version controlled. The analytical regime map reads threshold
248 constants from the same module as the classifier. The simulated backend is provided
249 solely for software testing and qualitative exploration; its outputs are excluded from
250 the reported measurements.

251 Statistics and reporting

252 This proof-of-concept study did not use random assignment, blinding, or a sample-size
253 calculation. Trial counts were fixed before summarisation. We report all measured grid
254 points in the machine-readable data, selected points in Table 1, arithmetic means, and
255 one standard deviation. No claim of statistical significance is made. Hardware-platform
256 generalisation requires independent machines and is left for future work.

257 Data availability

258 All data supporting the reported CPU measurements are included in the pub-
259 lic repository at <https://github.com/leemgs/orion>, under `code/results/cpu_probe/`
260 and `code/results/colab_probe/`. These directories contain the raw JSONL records
261 and machine-readable JSON summaries used in the text and table. No accelerator
262 data are reported in this article.

263 Code availability

264 The analysis and measurement code is available at <https://github.com/leemgs/orion>.
265 The `code/experiments/` directory contains the CPU probes, analytical-figure gener-
266 ator, and optional CUDA/XLA measurement paths. Exact reproduction commands
267 are provided in the README and Supplementary Information. A separately labelled
268 simulated backend is supplied for software testing; simulated output is not used as
269 evidence in this article.

270 Author Contributions

271 G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Val-
272 idation, Visualization, Writing – original draft, Writing – review & editing.

Competing Interests

The author declares no competing interests.

Acknowledgements

We thank Donghyeon Kang, Seungmin Oh and Yunsu Lee for valuable feedback. We are also grateful to Wooyeon Lee, Hyoyoung Kim, and Heekuk Lee, experts in systems and memory, whose insights and meaningful comments greatly improved this work.

References

- [1] Xiaoyu Ma and David Patterson. Challenges and research directions for large language model inference hardware. *IEEE Computer*, 59, 2026. URL <https://arxiv.org/abs/2601.05047>. Accepted.
- [2] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. Ai and memory wall. *IEEE Micro*, 2024. URL <https://arxiv.org/abs/2403.14123>.
- [3] International Energy Agency. Electricity 2024: Analysis and forecast to 2026. Technical report, IEA, 2024. URL <https://www.iea.org/reports/electricity-2024>. Global average carbon intensity: 494 gCO₂/kWh.
- [4] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proc. HPCA*, 2025. URL <https://arxiv.org/abs/2408.00741>.
- [5] Isabel Juniewicz. Hyperscaler capex is on trend to outpace their cash inflows by the end of 2026. Epoch AI, Data Insight, 2026. Accessed: 2026-06-21. [Online]. Available: <https://epoch.ai/data-insights/hyperscaler-capex-vs-cash-flow>.
- [6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [7] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL <https://arxiv.org/abs/2401.04088>.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proc. ICLR*, 2021. URL <https://arxiv.org/abs/2010.11929>.
- [9] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proc. ICML*, 2023. URL <https://arxiv.org/abs/2301.12597>.
- [10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. URL <https://arxiv.org/abs/1702.08734>.
- [11] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiaoping Zhang, Yuhan Dong, and Yu Wang. A survey on efficient inference for large

- language models. *arXiv preprint arXiv:2404.14294*, 2024. URL <https://arxiv.org/abs/2404.14294>.
- [12] Li Baolin et al. LLM inference serving: Survey of recent advances and opportunities. In *Proc. HPEC*, 2024. URL <https://arxiv.org/abs/2407.12391>.
- [13] Miao Xupeng et al. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58(1):1–37, 2025. URL <https://arxiv.org/abs/2312.15234>.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proc. OSDI*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/you>.
- [16] Du Kuntai et al. Prefillonly: An inference engine for prefill-only workloads in large language model applications. In *Proc. SOSP*, 2025. URL <https://arxiv.org/abs/2505.07203>.
- [17] Jin Yibo et al. P/d-serve: Serving disaggregated large language model at scale. *arXiv preprint arXiv:2408.08147*, 2024. URL <https://arxiv.org/abs/2408.08147>.
- [18] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2303.06865>.
- [19] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Chenyang Li, Dong Li, Erich Zheng, Olatunji Ruwase, and Yuxiong He. Deep-speed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022. URL <https://arxiv.org/abs/2207.00032>.
- [20] Xu Yi et al. Pie: Pooling CPU memory for LLM inference. *arXiv preprint arXiv:2411.09317*, 2024. URL <https://arxiv.org/abs/2411.09317>.
- [21] Xuanlin Jiang et al. Neo: Saving GPU memory crisis with CPU offloading for online llm inference. *arXiv preprint arXiv:2411.01142*, 2024. URL <https://arxiv.org/abs/2411.01142>.
- [22] Xiangwen Zhuge et al. SpecOffload: Unlocking latent GPU capacity for LLM inference on resource-constrained devices. *arXiv preprint arXiv:2505.10259*, 2025. URL <https://arxiv.org/abs/2505.10259>.
- [23] Xu Jiale et al. eLLM: Elastic memory management framework for efficient LLM serving. *arXiv preprint arXiv:2506.15155*, 2025. URL <https://arxiv.org/abs/2506.15155>.
- [24] Chien-Chin Huang, Gu Jin, and Jinyang Li. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the 25th*

- International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020. URL <https://doi.org/10.1145/3373376.3378530>.
- [25] Wang Kun et al. Swapnet: Efficient swapping for dnn inference on edge ai devices beyond the memory budget. *IEEE Transactions on Mobile Computing*, 23(9): 8935–8950, 2024. URL <https://arxiv.org/abs/2401.16757>.
- [26] Kida Shun et al. Revisiting memory swapping for big-memory applications. In *Proc. HPCASIA*, 2025. URL <https://doi.org/10.1145/3712031.3712332>.
- [27] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proc. ICML*, 2023. URL <https://arxiv.org/abs/2301.00774>.
- [28] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proc. NeurIPS*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [29] Korthikanti Vijay Anand et al. Reducing activation recomputation in large transformer models. *Proc. MLSys*, 5:341–353, 2023. URL <https://arxiv.org/abs/2205.05198>.
- [30] Huang Yanpin et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Proc. NeurIPS*, 2019. URL <https://arxiv.org/abs/1811.06965>.
- [31] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. URL <https://arxiv.org/abs/1909.08053>.
- [32] A.H. Hunter et al. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *Proc. OSDI*, 2021. URL <https://www.usenix.org/conference/osdi21/presentation/hunter>.
- [33] Yang Hanmei et al. Numalloc: A faster numa memory allocator. In *Proc. ISMM*, 2023. URL <https://doi.org/10.1145/3591195.3595276>.
- [34] Ugljesa Milic, Oreste Villa, Evgeny Bolotin, Akhil Arunkumar, Eiman Ebrahimi, Aamer Jaleel, Alex Ramirez, and David Nellans. Beyond the socket: NUMA-aware GPUs. In *Proc. MICRO*, pages 123–135, 2017. URL <https://doi.org/10.1145/3123939.3124534>.
- [35] Carl A. Waldspurger. Memory resource management in VMware ESX Server. In *Proceedings of the 5th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 181–194, 2002. URL <https://www.usenix.org/conference/osdi-02/memory-resource-management-vmware-esx-server>.
- [36] Long Sifan et al. A survey on intelligent network operations and performance optimization based on large language models. *IEEE Communications Surveys & Tutorials*, 2025. URL <https://ieeexplore.ieee.org/document/10829820>.
- [37] Zhi Wang et al. Model offloading techniques for edge AI: A survey. *IEEE Internet of Things Journal*, 2023. URL <https://doi.org/10.1109/JIOT.2023.3254417>. DOI: 10.1109/JIOT.2023.3254417.
- [38] Sarkar Rishov et al. Edge-moe: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. In *Proc. ICCAD*,

2023. URL <https://arxiv.org/abs/2305.18691>.
- [39] Luo Haoxiang et al. Toward edge general intelligence with multiple-large language model (Multi-LLM): architecture, trust, and orchestration. *IEEE Trans. Cogn. Commun. Netw.*, 2025. URL <https://arxiv.org/abs/2507.00672>.
- [40] He Zixuan et al. CPU–GPU heterogeneous computation offloading and resource allocation scheme for industrial internet of things. *IEEE Internet of Things Journal*, 11(6):11152–11164, 2023. URL <https://ieeexplore.ieee.org/document/10319405>.
- [41] Beaumont Olivier et al. Optimal GPU-CPU offloading strategies for deep neural network training. In *Proc. Euro-Par*, 2020. URL https://doi.org/10.1007/978-3-030-57675-2_10.
- [42] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970. doi: 10.1147/sj.92.0078. URL <https://doi.org/10.1147/sj.92.0078>.
- [43] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL <https://doi.org/10.1145/1498765.1498785>.
- [44] Leonard Kleinrock. *Queueing Systems, Volume 1: Theory*. John Wiley & Sons, New York, 1975. URL <https://dl.acm.org/doi/book/10.5555/1096491>. ISBN 978-0-471-49110-1.